

Autonomous Calendar Planner Agent

DS 593 - Theory and Applications of LLMs

Riya Bharamaraddi

1. Introduction

Students and busy professionals routinely face the overhead of manually organizing assignments, meetings, study sessions, and personal tasks across an increasingly fragmented set of commitments. The challenge is not a lack of calendar tools such as Google Calendar, Notion, and similar products, but that these tools require users to already know when to schedule something. They store what you tell them on their own, but don't offer help to decide when or how to fit a new commitment into an existing week.

This project builds an autonomous personal calendar planner agent that accepts natural-language task descriptions and automatically identifies available time slots, respects user preferences, and avoids scheduling conflicts. A user can type "Finish ML assignment by Friday, 2 hours, high priority, prefer mornings" and the system extracts the relevant structure, searches the calendar for a suitable slot, and confirms the scheduled event. The agent is evaluated against a first-come-first-served (FCFS) baseline to measure the concrete benefit of preference-aware and priority-aware scheduling.

The core question to be evaluated here is: does a hybrid LLM-plus-rule-based scheduling agent meaningfully outperform naive scheduling based on preference satisfaction and scheduling quality? The answer, supported by evaluation across 15 ground-truth scenarios, is yes — the agent achieves 91% preference satisfaction compared to 36% for the FCFS baseline, a 55-percentage-point improvement driven entirely by the agent's preference-aware slot selection logic.

2. Approach

2.1 System Architecture

The system follows a hybrid design: a large language model handles natural language understanding, while deterministic rule-based logic handles scheduling decisions. This separation was a deliberate design choice, as a fully LLM-driven scheduler would be flexible but unpredictable and difficult to evaluate (it might schedule the same input differently on successive runs). A fully rule-based system on the other hand would be reliable but would require rigid structured input. The hybrid approach captures the benefits of both.

We design the calendar planner as a ReAct-inspired agent that alternates between reasoning and acting steps. The LLM first interprets the user's natural language request and extracts structured task attributes such as duration, deadline, and preferences. A deterministic scheduler then acts on the current calendar state to identify feasible time slots and assign the task. If conflicts arise, the system re-enters the loop to revise constraints and reschedule until a valid schedule is produced.

The pipeline has four stages:

1. **Natural language input:** the user types a task description in free-form text.
2. **LLM parsing:** Groq's Llama 3.3 70B model extracts structured fields such as title, deadline, duration, priority, preferred time, and whether the task should be scheduled on a specific date versus before a deadline.
3. **Rule-based scheduling:** the scheduler searches available calendar slots using priority ordering, preferred time windows, and conflict detection.
4. **Output:** the confirmed event is added to the live calendar and displayed in the Streamlit

UI. 2.2 LLM Parser

The parser uses the Groq API with the llama-3.3-70b-versatile model. Groq was chosen over OpenAI for three reasons: the free tier provides 100,000 tokens per day which is sufficient for a course project as opposed to paying for tokens, inference speed is noticeably faster due to Groq's custom LPU hardware, and the llama-3.3-70b model performs comparably to GPT-4o-class models on structured extraction tasks (for a task like extracting title, deadline, and priority from a sentence, the quality difference between frontier and open-weight models is negligible).

Additionally, two prompting strategies were implemented and compared. Strategy A uses a single system prompt with a strict JSON schema and explicit rules for date resolution, priority normalization, and duration extraction. Strategy B uses a chain-of-thought approach where the model is instructed to reason through six sequential steps before producing the final JSON. Both strategies inject today's date into the prompt to enable reliable relative date resolution ("next Friday", "end of week"). The system prompt also distinguishes between "on [date]" (schedule exactly on that day) and "by [date]" (treat as a deadline), a distinction that significantly improves scheduling behavior for to-do style inputs.

The parser includes retry logic: if the first response cannot be parsed as valid JSON, a correction prompt is sent with the original input, the failed output, and the validation error.

2.3 Scheduling Engine

The scheduling engine implements four core functions: `get_events()`, `detect_conflict()`, `check_availability()`, and `schedule_event()`. Conflict detection uses an inclusive overlap check: two events conflict if neither ends before the other starts. `check_availability()` searches a given day in 30-minute increments starting from the preferred time window (8am for morning, 1pm for afternoon, 5pm for evening). `schedule_event()` controls which days are searched and in what order based on task priority:

- **High priority:** searches backwards from the deadline, maximizing the buffer of time available before the due date.
- **Medium priority:** starts three days before the deadline and falls back to earlier days if needed.
- **Low priority:** also searches backwards from the deadline, since low-urgency tasks should

not consume premium early slots.

As an addition, a task splitting function (`split_and_schedule()`) was added which handles tasks exceeding 2 hours when no single contiguous block is available. It divides the task into two equal chunks and schedules each on a different day, titling them "(part 1 of 2)" and "(part 2 of 2)".

2.4 Data

The primary data source is a synthetic calendar dataset of 98 events spanning six weeks (April 21 through June 5, 2026), representing a realistic student schedule. The calendar includes recurring lectures, gym sessions, and a varied set of one-off meetings, office hours, advisor meetings, lab sessions, and personal appointments. Events were manually constructed to create realistic scheduling pressure (mornings are partially blocked on most days, afternoons are busier mid-week, and evenings are mostly open).

Synthetic data was chosen over real calendar data for three reasons: reproducibility (the same scenarios produce the same ground-truth answers every run), privacy (no personal data is committed to a public repository), and controllability (the calendar can be designed to stress-test specific scheduling behaviors). Google Calendar API integration was considered but scoped out due to timeline constraints; it is identified as a natural extension in Section 6.

Evaluation uses 15 hand-designed ground-truth scenarios with known correct answers, defined before building the system. Each scenario specifies a natural-language task input, a set of existing calendar events, and an expected output including deadline, duration, priority, preferred time, and conflict constraints. Scenarios span five categories: preference satisfaction, urgency override, priority ordering, conflict navigation, and parser stress tests with vague or ambiguous language.

3. Evaluation

3.1 Metrics

Three primary metrics were measured across all 15 scenarios for both the agent and the FCFS baseline:

Scheduling accuracy: the percentage of scenarios where the scheduled slot satisfies all constraints — correct duration, no conflicts with blocked events, and completion before the deadline.

Preference satisfaction: among scenarios with a stated time preference (morning, afternoon, or evening), the percentage where the scheduled start time falls within the expected window.

Conflict-free rate: the percentage of scheduled events that do not overlap with any blocked event in the scenario, regardless of other constraints.

A fourth metric, parser accuracy, was measured separately for the two prompting strategies across priority extraction, duration extraction, and preferred time extraction.

3.2 Baseline

The FCFS baseline schedules tasks in arrival order with no preference awareness. It always searches forward from today and takes the absolute first available slot starting at work_start_hour (8am), ignoring preferred_time entirely. The baseline uses the same detect_conflict() logic as the agent, so both are guaranteed to produce conflict-free schedules — the only difference is where within the available time they choose to place events. This design isolates the preference-aware scheduling logic as the variable being tested.

3.3 Prompting Strategy Comparison

Strategy A (single-prompt) and Strategy B (chain-of-thought) were compared on parser accuracy across all 15 scenarios. The comparison measures per-field accuracy for priority, duration, and preferred time extraction, as well as overall accuracy requiring all fields to be correct simultaneously.

4. Results and Discussion

4.1 Main Results

Metric	Agent	FCFS Baseline
Scheduling accuracy	100%	100%
Preference satisfaction	91%	36%
Conflict-free rate	100%	100%

Table 1. Agent vs. FCFS baseline across 15 ground-truth scenarios.

Both schedulers achieve 100% scheduling accuracy and conflict-free rate because detect_conflict() is shared logic so neither will ever place an event on top of an existing one, and on a calendar with realistic but not extreme density, both always find a valid slot before the deadline. The meaningful result is preference satisfaction: the agent honors user time preferences in 91% of cases compared to 36% for the FCFS baseline, showcasing a significant difference.

The 36% FCFS preference satisfaction is not zero because the baseline occasionally stumbles into the preferred time window by accident (e.g., if the first available slot at 8am happens to be in the morning and the user requested morning, FCFS gets credit). The gap between 91% and 36% represents the agent's active preference-seeking behavior: it starts searching from the preferred time window and only falls back to non-preferred slots when the preferred window is fully blocked.

4.2 Iteration History

The evaluation system went through three iterations before reaching stable results. In the first run, the batch evaluator used exact string matching for task titles. The LLM dropped the word "Finish" from "Finish problem set 3", producing "problem set 3" which failed the check. This is a fundamental limitation of exact-match evaluation for LLM outputs and motivated a switch to substring matching. In the second run, scenario 5 failed because the scheduler treated a same-day deadline as having no remaining time. This was fixed by making the deadline boundary inclusive and searching remaining hours on the current day. After both fixes, the system reached 5/5 on the original 5-scenario set, and 13/15 (87%) on the expanded 15-scenario set.

The FCFS baseline produced identical preference satisfaction to the agent (both 91%) in early evaluation runs because `baseline_fcfs_scheduler` was internally calling `check_availability()` with the task's `preferred_time` merged in, making it preference-aware despite being labeled as FCFS. Correcting this to use a blind `UserPreferences` object with `preferred_time=None` rectified that.

4.3 Prompting Strategy Comparison

Field	Strategy A	Strategy B
Priority accuracy	93%	100%
Duration accuracy	100%	100%
Preferred time accuracy	80%	87%

Table 2. Parser accuracy comparison between Strategy A (single-prompt) and Strategy B (chain-of-thought).

Strategy B (chain-of-thought) outperforms Strategy A on priority and preferred time accuracy. A notable difference lies in ambiguous priority language: inputs like "pretty important" mapped to high priority under Strategy A (over-interpretation) and to medium under Strategy B (more conservative and correct). This suggests that explicitly prompting the model to reason through urgency mapping before committing to a value produces more calibrated outputs.

Strategy A was retained as the default for this case (`parse_task` calls `parse_task_strategy_a`) because it has lower latency and is sufficient for unambiguous inputs which represent the majority of real-world inputs. Strategy B can be used in cases requiring higher accuracy on vague language.

4.4 Known Failure Modes

Two consistent failure modes were observed. First, the parser defaults to medium priority when no urgency signal is present in the input ("Prepare slides for the lab update" with no priority stated). Medium is a reasonable default but the system has no mechanism to ask for clarification, so it might be

better for a production system to prompt the user rather than assume. Second, relative date resolution occasionally produces inconsistent results across runs. "End of week" resolved to both Saturday and Sunday depending on model state, suggesting the model does not have a stable convention for week boundaries. Injecting today's date into the system prompt reduced this significantly but did not eliminate it entirely.

5. Ethics and Limitations

In this project, scheduling is performed on synthetic data, so no real personal information is processed. A deployed version of this system would handle sensitive data: meeting titles, location patterns, health appointments, and behavioral patterns inferred from scheduling habits. This warrants strong access controls, local-first storage where possible, and clear user consent for any cloud processing.

The LLM infers priority from natural language phrasing. Phrasing conventions vary across cultures, neurodivergent communication styles, and professional contexts. A user who communicates urgency differently than the training distribution expects may find their tasks systematically under-prioritized. This is difficult to audit without diverse user testing.

The current implementation has no concept of shared calendars, meeting coordination, or multi-stakeholder scheduling. Extending to shared contexts introduces consent and access questions that this architecture does not address.

All evaluation was performed on a single synthetic calendar representing one student's schedule. The system has not been tested on diverse schedule patterns, different academic calendars, or professional rather than student contexts. Generalization claims are therefore limited.

6. Future Work

The system was designed with Google Calendar integration in mind (agent/calendar_api.py scaffolds the OAuth2 flow and read/write functions), but live integration was scoped out due to limited time. Connecting to real calendar data would enable evaluation on genuine scheduling diversity.

The recurring field is stored in the Event dataclass and JSON but is not used to auto-generate future occurrences. A production system would expand recurring rules dynamically rather than requiring manual duplication of events across weeks.

The current parser makes a single inference pass and commits to an output. A conversational agent could ask follow-up questions when inputs are ambiguous, such as "Did you mean high or medium priority?" before scheduling. This would reduce priority inference errors on vague inputs.

15 scenarios is sufficient to demonstrate proof of concept but not ideal to make strong statistical claims. A larger and more diverse test set that ideally includes real anonymized scheduling requests would produce more reliable accuracy estimates.

7. References

- [1] Groq. (2024). Groq API Documentation. <https://console.groq.com/docs>
- [2] Meta AI. (2024). Llama 3.3: Open Foundation and Fine-Tuned Chat Models. <https://ai.meta.com/llama/>
- [3] Streamlit. (2024). Streamlit Documentation. <https://docs.streamlit.io>
- [4] Google. (2024). Google Calendar API Documentation. <https://developers.google.com/calendar>
For informational purposes as it wasn't implemented.
- [5] Wei, J., et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022.
- [6] Python Software Foundation. python-dateutil library. <https://dateutil.readthedocs.io>
- [7] Anthropic. (2024). Claude AI. used for code generation assistance and debugging throughout development.
- [8] OpenAI Codex. (2024). Used as coding assistant for scaffolding agent/scheduler.py and eval/[evaluate.py](#) as well as implementing strategies.